

TOPIC 4 : DYNAMIC PROGRAMMING

Q1 – Dice Throw Problem

Aim

To implement the **Dice Throw Problem** using **Dynamic Programming in Python** and calculate the number of ways to obtain the given target sum.

Algorithm

1. Create a DP table where $dp[d][s]$ represents the number of ways to obtain sum s using d dice.
2. Initialize $dp[0][0] = 1$.
3. For each die, consider every possible sum.
4. Try every face value from 1 to num_sides .
5. Add the number of ways to obtain the remaining sum using the previous dice.
6. Continue until all dice are processed.
7. The answer is $dp[num_dice][target]$.

Python Source Code

```
File Edit Format Run Options Window Help
def dice_throw(num_sides, num_dice, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]

    dp[0][0] = 1

    for dice in range(1, num_dice + 1):
        for total in range(1, target + 1):
            for face in range(1, num_sides + 1):
                if total >= face:
                    dp[dice][total] += dp[dice - 1][total - face]

    return dp[num_dice][target]

num_sides = 6
num_dice = 2
target = 7

result = dice_throw(num_sides, num_dice, target)

print("Number of ways to reach sum", target, ":", result)
```

```
===== RESTART: C:/Users/HP/Download  
Number of ways to reach sum 7 : 6
```

Time Complexity

$O(\text{num_dice} \times \text{target} \times \text{num_sides})$

Space Complexity

$O(\text{num_dice} \times \text{target})$

Result

Thus, the **Dice Throw Problem** was successfully solved using **Dynamic Programming in Python**, and the number of ways to reach the target sum was calculated correctly.

Q2 – Assembly Line Scheduling

Aim

To find the minimum processing time for a product passing through two assembly lines using **Dynamic Programming in Python**.

Algorithm

1. Initialize the processing time for the first station of both lines by adding their entry times.
2. For each subsequent station:
 - Calculate the time if the product stays on the same line.
 - Calculate the time if the product transfers from the other line.
3. Choose the minimum of these two possibilities.
4. Add the processing time of the current station.
5. After the last station, add the corresponding exit time.
6. The smaller of the two final times is the minimum processing time.

Python Source Code

```
def assembly_line_scheduling(a1, a2, t1, t2, e1, e2, x1, x2):
    n = len(a1)

    dp1 = [0] * n
    dp2 = [0] * n

    dp1[0] = e1 + a1[0]
    dp2[0] = e2 + a2[0]

    for i in range(1, n):
        dp1[i] = min(
            dp1[i - 1] + a1[i],
            dp2[i - 1] + t2[i - 1] + a1[i]
        )

        dp2[i] = min(
            dp2[i - 1] + a2[i],
            dp1[i - 1] + t1[i - 1] + a2[i]
        )

    final1 = dp1[n - 1] + x1
    final2 = dp2[n - 1] + x2

    return min(final1, final2)

a1 = [4, 5, 3, 2]
a2 = [2, 10, 1, 4]

t1 = [7, 4, 5]
t2 = [9, 2, 8]

e1 = 10
e2 = 12

x1 = 18
x2 = 7

result = assembly_line_scheduling(a1, a2, t1, t2, e1, e2, x1, x2)

print("Minimum processing time:", result)
```

OUTPUT:

```
----- RESTART: C:/Users/
Minimum processing time: 35
|
```

Time Complexity

$O(n)$

Each station is processed once.

Space Complexity

$O(n)$

Two DP arrays are used.

Result

Thus, the **Assembly Line Scheduling problem was successfully implemented using Dynamic Programming in Python** to determine the minimum time required to process the product from entry to exit.

Aim

To find the **minimum production time** for processing tasks across three assembly lines using **Dynamic Programming in Python**, while considering transfer times and task dependencies.

Given Input

Number of stations:

3

Station times:

Line 1: [5, 9, 3]

Line 2: [6, 8, 4]

Line 3: [7, 6, 5]

Transfer times:

[0, 2, 3]

[2, 0, 4]

[3, 4, 0]

Dependencies:

[(0, 1), (1, 2)]

The input values above are taken directly from Q3.

Pasted text

Algorithm

1. Create a DP table for the three assembly lines.
2. Initialize the first station with its processing time.
3. For every next station, calculate the time for each possible current line.
4. Consider:
 - Continuing on the same line.
 - Transferring from another line.
5. Add the corresponding transfer time when changing lines.
6. Choose the minimum time for each state.
7. Continue until the final station.
8. Select the minimum time among the three lines.
9. Respect the sequential dependencies between stations.

PYTHON SOURCE CODE:

```
File Edit Format Run Options Window Help
lines = len(station_times)
stations = len(station_times[0])

dp = [[0] * stations for _ in range(lines)]

for i in range(lines):
    dp[i][0] = station_times[i][0]

for j in range(1, stations):
    for i in range(lines):
        minimum = float('inf')

        for previous_line in range(lines):
            time = dp[previous_line][j - 1]

            if previous_line != i:
                time += transfer_times[previous_line][i]

            time += station_times[i][j]

            minimum = min(minimum, time)

        dp[i][j] = minimum

    return min(dp[i][stations - 1] for i in range(lines))

station_times = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5]
]

transfer_times = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0]
]

result = minimum_production_time(station_times, transfer_times)

print("Minimum production time:", result)
```

OUTPUT:

```
===== RESTART: C:/Users/HP/
Minimum production time: 17
> |
```

Time Complexity

$O(L^2 \times n)$

where:

- L = number of assembly lines
- n = number of stations

For 3 lines, this is effectively $O(n)$.

Space Complexity

$O(L \times n)$

for the DP table.

Result

Thus, the **three-assembly-line scheduling problem was implemented using Dynamic Programming in Python**, considering station processing times and transfer times to determine the minimum production time. The dependency information given in Q3 is sequential: the output of station 1 is required before station 2, and station 2 before station 3.

Q4 – Minimum Path Distance Using Matrix

Q4 asks you to find the **minimum path distance using a matrix representation**. The question provides three test cases with expected outputs **80, 40, and 12**.

Pasted text

Aim

To implement a Python program using **Dynamic Programming** to find the minimum path distance represented by a cost matrix.

Algorithm

1. Read the distance matrix.
2. Initialize a DP matrix with the same values.
3. Start from the first row and first column.
4. For every cell, calculate the minimum cost of reaching it.
5. Add the current matrix value to the minimum available previous path.
6. Continue until the destination cell is reached.
7. The final DP value gives the minimum path distance.

Python Source Code

```
File Edit Format Run Options Window Help
def minimum_path_distance(matrix):
    n = len(matrix)
    dp = [[0] * n for _ in range(n)]

    dp[0][0] = matrix[0][0]

    for i in range(1, n):
        dp[i][0] = dp[i - 1][0] + matrix[i][0]

    for j in range(1, n):
        dp[0][j] = dp[0][j - 1] + matrix[0][j]

    for i in range(1, n):
        for j in range(1, n):
            dp[i][j] = matrix[i][j] + min(dp[i - 1][j], dp[i][j - 1])

    return dp[n - 1][n - 1]

matrix = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

result = minimum_path_distance(matrix)
print("Minimum path distance:", result)
```

OUTPUT:

```
===== RESTART:| C:/Use
Minimum path distance: 75
```

Time Complexity

$O(n^2)$

Each element of the matrix is processed once.

Space Complexity

$O(n^2)$

The DP matrix requires $n \times n$ space.

Result

Thus, the **minimum path distance** was calculated using **Dynamic Programming in Python** for the given matrix representatio

Q5 – Traveling Salesperson Problem (TSP)

Aim

To find the shortest possible route that visits every city exactly once and returns to the starting city using **Dynamic Programming**.

Algorithm

1. Represent the cities using a distance matrix.
2. Select city A as the starting city.
3. Create a DP table using subsets of visited cities.
4. For each subset, calculate the minimum cost of reaching every possible last city.
5. Add the distance from the last city back to A.
6. Compare all possible complete tours.
7. Select the route with the minimum total distance.
8. Display the shortest route and its total distance.

Python Source Code

```
full_mask = (1 << n) - 1
best_cost = float("inf")
last_city = None

for city in range(1, n):
    cost = dp[(full_mask, city)] + distance[city][0]

    if cost < best_cost:
        best_cost = cost
        last_city = city

route = []
mask = full_mask
current = last_city

while current != 0:
    route.append(current)
    previous = parent[(mask, current)]
    mask ^= 1 << current
    current = previous

route.append(0)
route.reverse()
route.append(0)

return route, best_cost

cities = ["A", "B", "C", "D", "E"]

distance = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 30],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 15],
    [25, 30, 20, 15, 0]
]

route, cost = tsp(distance)

print("Shortest Route:", " -> ".join(cities[i] for i in route))
print("Total Distance:", cost)
```

```
===== RESTART: C:/Users/HP/Downloads/D
Shortest Route: A -> C -> E -> D -> B -> A
Total Distance: 85
> |
```

Time Complexity

$O(n)$

Each step is processed once.

Space Complexity

$O(n)$

The DP array stores the minimum cost for each position.

Result

Thus, the **Minimum Cost Climbing Stairs problem** was successfully implemented using **Dynamic Programming in Python**, and the minimum cost required to reach the top was obtained as **15**.

Q6 – Longest Palindromic Substring

Aim

To find the **longest palindromic substring** in a given string using **Dynamic Programming in Python**.

Algorithm

1. Create a DP table where $dp[i][j]$ represents whether the substring from index i to j is a palindrome.
2. Every single character is a palindrome.
3. Check substrings of increasing length.
4. A substring is a palindrome if:
 - Its first and last characters are equal.
 - The characters inside it form a palindrome.
5. Whenever a longer palindrome is found, store its starting position and length.
6. Return the longest palindromic substring.

Python Source Code

```
def longest_palindrome(s):
    n = len(s)

    if n < 2:
        return s

    dp = [[False] * n for _ in range(n)]

    start = 0
    max_length = 1

    for i in range(n):
        dp[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2 or dp[i + 1][j - 1]:
                    dp[i][j] = True

                    if length > max_length:
                        start = i
                        max_length = length

    return s[start:start + max_length]

s = "babad"

result = longest_palindrome(s)

print("Longest Palindromic Substring:", result)
```

Longest Palindromic Substring: bab

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Result

Thus, the **Longest Palindromic Substring** was successfully found using **Dynamic Programming in Python**. For "babad", the program returns "bab".

Q7 – Longest Substring Without Repeating Characters

Aim

To find the length of the longest substring containing **no repeated characters** using Python.

Algorithm

1. Use a sliding window to keep track of the current substring.
2. Maintain the last position of every character.
3. Traverse the string from left to right.
4. If a character is repeated inside the current window, move the left boundary.
5. Calculate the length of the current window.
6. Keep track of the maximum length.
7. Return the maximum length.

Python Source Code

```
def longest_substring(s):
    last_position = {}
    left = 0
    max_length = 0

    for right in range(len(s)):
        if s[right] in last_position and last_position[s[right]] >= left:
            left = last_position[s[right]] + 1

        last_position[s[right]] = right

        max_length = max(max_length, right - left + 1)

    return max_length

s = "abcabcbb"
result = longest_substring(s)
print("Length of longest substring:", result)
```

```
----- RESTART: C:/Users/III
Length of longest substring: 3
|
```

Time Complexity

O(n)

Each character is processed at most a constant number of times.

Space Complexity

O(n)

The dictionary stores the last position of characters.

Result

Thus, the **longest substring without repeating characters** was successfully found using Python. For "abcabcbb", the maximum length is **3**.

Q6 – Minimum Cost to Reach the Top

Aim

To find the **minimum cost required to reach the top of a staircase**, where each step has an associated cost, using **Dynamic Programming in Python**.

Algorithm

1. Store the cost of each step in an array.
2. Create a DP array to store the minimum cost required to reach each step.
3. Initialize the first two positions.
4. For every step, calculate the minimum cost of reaching it from either of the previous two steps.
5. Add the current step's cost.
6. At the end, take the minimum cost of reaching the top from the last two positions.
7. Display the minimum cost.

Python Source Code

```
def longest_palindrome(s):
    n = len(s)

    if n < 2:
        return s

    dp = [[False] * n for _ in range(n)]

    start = 0
    max_length = 1

    for i in range(n):
        dp[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if s[i] == s[j]:
                if length == 2 or dp[i + 1][j - 1]:
```

```

        dp[i][j] = True

        if length > max_length:
            start = i
            max_length = length

    return s[start:start + max_length]

s = "babad"
result = longest_palindrome(s)
print("Longest Palindromic Substring:", result)

```

OUTPUT:

```

===== RESTART: C:/Users/HP/Downlo
Longest Palindromic Substring: bab

```

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Result

Thus, the **Longest Palindromic Substring** was successfully found using **Dynamic Programming in Python**. For "babad", the program returns "bab".

Q7 – Longest Substring Without Repeating Characters

Q7 asks you to find the **length of the longest substring without repeating characters**.

Pasted text(5)

Aim

To find the length of the longest substring containing **no repeated characters** using Python.

Algorithm

1. Use a sliding window to keep track of the current substring.
2. Maintain the last position of every character.
3. Traverse the string from left to right.
4. If a character is repeated inside the current window, move the left boundary.
5. Calculate the length of the current window.
6. Keep track of the maximum length.
7. Return the maximum length.

Python Source Code

```
file  edit  format  run  options  window  help
def longest_substring(s):
    last_position = {}
    left = 0
    max_length = 0

    for right in range(len(s)):
        if s[right] in last_position and last_position[s[right]] >= left:
            left = last_position[s[right]] + 1

        last_position[s[right]] = right

        max_length = max(max_length, right - left + 1)

    return max_length

s = "abcabcbb"
result = longest_substring(s)
print("Length of longest substring:", result)
```

```
===== RESTART: C:/Users/HP
Length of longest substring: 3
>>
```

Time Complexity

O(n)

Each character is processed at most a constant number of times.

Space Complexity

O(n)

The dictionary stores the last position of characters.

Result

Thus, the **longest substring without repeating characters** was successfully found using Python. For "abcabcbb", the maximum length is **3**.

Q9 – Word Break

Aim

To determine whether an input string can be segmented into a sequence of words from a given dictionary using **Dynamic Programming in Python**.

Algorithm

1. Store all dictionary words.
2. Create a DP array of size $n + 1$.
3. Set $dp[0] = \text{True}$.
4. Check every possible substring of the input string.
5. If the previous part can be segmented and the current substring is in the dictionary, mark the current position as True.
6. Continue until the entire string is processed.
7. If $dp[n]$ is True, print **Yes**; otherwise print **No**.

Python Source Code

File Edit Format Run Options Window Help

```
def word_break(s, word_dict):
    word_set = set(word_dict)
    n = len(s)

    dp = [False] * (n + 1)
    dp[0] = True

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[n]

word_dict = [
    "i", "like", "sam", "sung", "samsung",
    "mobile", "ice", "cream", "icecream",
    "man", "go", "mango"
]

s = "ilike"

result = word_break(s, word_dict)

if result:
    print("Yes")
else:
    print("No")
```

OUTPUT:

```
===== RESTART: C:/Users/HP/I
Yes
>
```

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Result

Thus, the input string was successfully checked for dictionary-based segmentation using **Dynamic Programming in Python**.

Q10 – Text Justification

Aim

To format a given list of words into lines of exactly maxWidth characters using **greedy text justification**.

Algorithm

1. Start from the first word.
2. Add as many words as possible to the current line without exceeding maxWidth.
3. Calculate the number of spaces required.
4. If it is not the last line and contains more than one word, distribute spaces evenly between words.
5. If extra spaces remain, place them from left to right.
6. For the last line or a line containing one word, use left justification.
7. Continue until all words are processed.

Python Source Code

```
def full_justify(words, maxWidth):
    result = []
    i = 0

    while i < len(words):
        line = [words[i]]
        line_length = len(words[i])
        i += 1

        while i < len(words) and line_length + 1 + len(words[i]) <= maxWidth:
            line.append(words[i])
            line_length += 1 + len(words[i])
            i += 1

        total_spaces = maxWidth - sum(len(word) for word in line)

        if i == len(words) or len(line) == 1:
            result.append(" ".join(line) + " " * (maxWidth - len(" ".join(line))))
        else:
            gaps = len(line) - 1
            space_each = total_spaces // gaps
            extra_spaces = total_spaces % gaps

            current_line = ""

            for j in range(gaps):
                current_line += line[j]
                current_line += " " * (space_each + (1 if j < extra_spaces else 0))

            current_line += line[-1]
            result.append(current_line)

    return result

words = ["This", "is", "an", "example", "of", "text", "justification."]
maxWidth = 16

result = full_justify(words, maxWidth)
print("Justified Text:")
```

OUTPUT:

```
>> yes
>> ===== RESTART: C:/Us
Justified Text:
"This    is    an"
"example of text"
"justification. "
>>
```

Time Complexity

O(n) approximately, where n is the total number of characters in the input words.

Space Complexity

O(n) for storing the formatted result.

Result

Thus, the **Text Justification problem was successfully implemented using Python**, producing lines with exactly maxWidth characters while following the required greedy packing and space-distribution rules.

Q11 – WordFilter (Prefix and Suffix Search)

Aim

To implement a special dictionary that searches for a word having a given **prefix and suffix**, and returns the **largest index** of the matching word.

Algorithm

1. Store all the words with their indices.
2. For every word, generate all possible prefix-suffix combinations.
3. Store each combination along with the word's index.
4. For repeated combinations, keep the largest index.
5. In f(pref, suff), combine the prefix and suffix.
6. Search for the combination in the dictionary.
7. Return its largest index.
8. If no matching word exists, return -1.

Python Source Code

```
class WordFilter:

    def __init__(self, words):
        self.data = {}

        for index, word in enumerate(words):
            for i in range(len(word) + 1):
                prefix = word[:i]

                for j in range(len(word) + 1):
                    suffix = word[j:]
                    self.data[prefix + "#" + suffix] = index

    def f(self, pref, suff):
        return self.data.get(pref + "#" + suff, -1)

words = ["apple"]

wordFilter = WordFilter(words)

print(wordFilter.f("a", "e"))
```

```
> |
  | ===== RESTART: C:/User
  | 0
  |
> |
```

Time Complexity

Construction: $O(n \times L^2)$

where n is the number of words and L is the maximum word length.

Search: $O(1)$ average time.

Space Complexity

$O(n \times L^2)$

Result

Thus, the **WordFilter** prefix and suffix search problem was successfully implemented in **Python**.

Q12 – Floyd's Algorithm

Aim

To implement **Floyd's Algorithm** to find the shortest paths between all pairs of cities and display the distance matrix before and after applying the algorithm.

Algorithm

1. Create a distance matrix for all cities.
2. Initialize the distance between a city and itself as 0.
3. Initialize directly connected cities with their edge weights.
4. Set the distance between unconnected cities to infinity.
5. Consider each city as an intermediate city.
6. For every pair of cities (i, j), check whether going through the intermediate city gives a shorter path.
7. Update the distance matrix if a shorter path is found.
8. After all intermediate cities are considered, display the final shortest-distance matrix.
9. Count the cities within the given distance threshold.
10. If two cities have the same count, select the city with the larger index.

Python Source Code

```
def floyd_warshall(n, edges, threshold):
    INF = float('inf')

    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    print("Distance Matrix Before Floyd's Algorithm:")
    for row in dist:
        print(row)
```

```

for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k] != INF and dist[k][j] != INF:
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

print("\nDistance Matrix After Floyd's Algorithm:")
for row in dist:
    print(row)

min_count = float('inf')
result = -1

for i in range(n):
    count = 0

    for j in range(n):
        if i != j and dist[i][j] <= threshold:
            count += 1

    if count <= min_count:
        min_count = count
        result = i

return dist, result

n = 4

edges = [
    [0, 1, 3],
    [1, 2, 1],
    [1, 3, 4],
    [2, 3, 1]
]

distance_threshold = 4

dist, result = floyd_warshall(n, edges, distance_threshold)

print("\nCity with required minimum neighbors:", result)

```

OUTPUT:

```
===== RESTART: C:/Users/HP/Download
Distance Matrix Before Floyd's Algorithm:
[0, 3, inf, inf]
[3, 0, 1, 4]
[inf, 1, 0, 1]
[inf, 4, 1, 0]

Distance Matrix After Floyd's Algorithm:
[0, 3, 4, 5]
[3, 0, 1, 2]
[4, 1, 0, 1]
[5, 2, 1, 0]

City with required minimum neighbors: 3
> |
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, **Floyd's Algorithm** was successfully implemented in **Python** to calculate the shortest distances between all pairs of cities. For the given test case, the required output is **City 3**.

Q13 – Floyd's Algorithm with Link Failure

Aim

To implement **Floyd's Algorithm** to calculate the shortest paths between all pairs of routers and determine the shortest path from **Router A to Router F** before and after the failure of the link between **Router B and Router D**.

Algorithm

1. Create a distance matrix for the six routers.
2. Initialize the diagonal elements to 0.
3. Add the given link distances to the matrix.

4. Apply Floyd-Warshall Algorithm to find all-pairs shortest paths.
5. Find the shortest distance from Router A to Router F.
6. Remove the link between Router B and Router D.
7. Recalculate the distance matrix using Floyd-Warshall Algorithm.
8. Find the new shortest distance from Router A to Router F.
9. Display both distances.

Python Source Code

```
def floyd_warshall(n, edges):
    INF = float('inf')

    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )

    return dist
```

```

routers = ["A", "B", "C", "D", "E", "F"]

edges = [
    [0, 1, 1],
    [0, 2, 5],
    [1, 2, 2],
    [1, 3, 1],
    [2, 4, 3],
    [3, 4, 1],
    [3, 5, 6],
    [4, 5, 2]
]

dist_before = floyd_warshall(6, edges)

print("Shortest path from Router A to Router F before link failure:",
      dist_before[0][5])
```

```

updated_edges = [
    [0, 1, 1],
    [0, 2, 5],
    [1, 2, 2],
    [2, 4, 3],
    [3, 4, 1],
    [3, 5, 6],
    [4, 5, 2]
]

dist_after = floyd_warshall(6, updated_edges)

print("Shortest path from Router A to Router F after link failure:",
      dist_after[0][5])
```

OUTPUT:

```
===== RESTART: C:/Users/HP/Downloads/Documents/e1.py =====
Shortest path from Router A to Router F before link failure: 5
Shortest path from Router A to Router F after link failure: 8
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, Floyd's Algorithm was successfully implemented to calculate the shortest paths between all routers. Before the **B–D link failure**, the shortest distance from **A to F is 5**. After removing that link, the program recalculates the shortest path.

Q14 – Floyd's Algorithm

Aim

To implement **Floyd's Algorithm** to find the shortest paths between all pairs of cities and determine the required shortest path using the given graph.

Algorithm

1. Create a distance matrix for all 5 cities.
2. Initialize the diagonal elements to 0.
3. Insert the given edge distances into the matrix.
4. Set the distance of unconnected cities to infinity.
5. Apply Floyd-Warshall Algorithm.
6. Use each city as an intermediate city.
7. Update the distance between every pair if a shorter path is found.
8. Display the distance matrix before and after the algorithm.
9. Count the cities reachable within the given threshold.
10. For equal counts, select the city with the greatest index.

Python Source Code

```
def floyd_warshall(n, edges, threshold):
    INF = float('inf')

    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    print("Distance Matrix Before Floyd's Algorithm:")
    for row in dist:
        print(row)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )

    print("\nDistance Matrix After Floyd's Algorithm:")
    for row in dist:
        print(row)

    min_count = float('inf')
    result = -1

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1

        if count <= min_count:
            min_count = count
```

```

        result = i

    return dist, result

n = 5
edges = [
    [0, 1, 2],
    [0, 4, 8],
    [1, 2, 3],
    [1, 4, 2],
    [2, 3, 1],
    [3, 4, 1]
]

distance_threshold = 2

dist, result = floyd_warshall(
    n,
    edges,
    distance_threshold
)

print("\nCity with the required minimum number of neighbors:", result)

```

OUTPUT:

```

===== RESTART: C:/Users/HP/Downloads/Documents/
Distance Matrix Before Floyd's Algorithm:
[0, 2, inf, inf, 8]
[2, 0, 3, inf, 2]
[inf, 3, 0, 1, inf]
[inf, inf, 1, 0, 1]
[8, 2, inf, 1, 0]

Distance Matrix After Floyd's Algorithm:
[0, 2, 5, 5, 4]
[2, 0, 3, 3, 2]
[5, 3, 0, 1, 2]
[5, 3, 1, 0, 1]
[4, 2, 2, 1, 0]

City with the required minimum number of neighbors: 0

```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Result

Thus, Floyd's Algorithm was successfully implemented in Python to find the shortest paths between all pairs of cities. For the main Q14 test case, the required city is City 0.

Q15 – Optimal Binary Search Tree (OBST)

Aim

To construct an Optimal Binary Search Tree for the given keys and frequencies using Dynamic Programming, and calculate its minimum search cost.

Algorithm

1. Store the keys and their search frequencies.
2. Create a cost table $\text{cost}[i][j]$.
3. For a single key, the cost is its frequency.
4. For every possible range of keys:
 - Calculate the total frequency.
 - Try each key as the root.
 - Calculate the cost of the left and right subtrees.
5. Choose the root that gives the minimum cost.
6. Store the selected root in the root table.
7. The value $\text{cost}[0][n-1]$ gives the minimum OBST cost.
8. Recursively construct the tree using the root table.

Python Source Code

```
def optimal_bst(keys, freq):
    n = len(keys)

    cost = [[0] * n for _ in range(n)]
    root = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            total_freq = sum(freq[i:j + 1])
            cost[i][j] = float('inf')

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                current_cost = left + right + total_freq

                if current_cost < cost[i][j]:
                    cost[i][j] = current_cost
                    root[i][j] = r

    def build_tree(i, j):
        if i > j:
            return None

        r = root[i][j]

        return {
            "key": keys[r],
            "left": build_tree(i, r - 1),
            "right": build_tree(r + 1, j)
        }

    tree = build_tree(0, n - 1)

    return cost, root, tree
```

```

keys = ["A", "B", "C", "D"]
freq = [0.1, 0.2, 0.4, 0.3]

cost, root, tree = optimal_bst(keys, freq)

print("Minimum Cost:", cost[0][len(keys) - 1])

print("\nCost Matrix:")
for row in cost:
    print(row)

print("\nRoot Matrix:")
for row in root:
    print(row)

print("\nRoot of Optimal BST:", tree["key"])

```

```

===== RESTART: C:/Users/HP/De
Minimum Cost: 1.7

Cost Matrix:
[0.1, 0.4, 1.1, 1.7]
[0, 0.2, 0.8, 1.4]
[0, 0, 0.4, 1.0]
[0, 0, 0, 0.3]

Root Matrix:
[0, 1, 2, 2]
[0, 1, 2, 2]
[0, 0, 2, 2]
[0, 0, 0, 3]

Root of Optimal BST: C

```

Time Complexity **$O(n^3)$** **Space Complexity** **$O(n^2)$** **Result**

Thus, the **Optimal Binary Search Tree** was successfully implemented using **Dynamic Programming in Python**. For keys A, B, C, D with frequencies 0.1, 0.2, 0.4, 0.3, the minimum OBST cost is **1.7**.

Q16 – Subset Generation

Q16 asks you to generate **all subsets containing the element 3** and also gives the standard power-set examples for unique integers.

Pasted text(4)

Aim

To generate all possible subsets of a given set that **contain a specified element x**, using Python.

Algorithm

1. Start with an empty list to store the subsets.
2. Generate all possible subsets using backtracking.
3. For every element, choose either:
 - Include the element.
 - Exclude the element.
4. After generating a subset, check whether it contains x.
5. If it contains x, add it to the result.
6. Continue until all possible subsets are generated.
7. Return the list of valid subsets.

Python Source Code

```
def subsets_containing_element(nums, x):  
    result = []  
  
    def backtrack(index, current):  
        if index == len(nums):  
            if x in current:  
                result.append(current.copy())  
            return  
  
        current.append(nums[index])  
        backtrack(index + 1, current)  
  
        current.pop()  
        backtrack(index + 1, current)  
  
    backtrack(0, [])  
  
    return result  
  
E = [2, 3, 4, 5]  
x = 3  
  
result = subsets_containing_element(E, x)  
  
print("Subsets containing", x, ":")  
  
for subset in result:  
    print(subset)|
```

```
===== RESTART:
Subsets containing 3
[2, 3, 4, 5]
[2, 3, 4]
[2, 3, 5]
[2, 3]
[3, 4, 5]
[3, 4]
[3, 5]
[3]
> |
```

Time Complexity

$O(2^n \times n)$

There are 2^n subsets and copying a subset can take up to $O(n)$ time.

Space Complexity

$O(2^n \times n)$

The result stores all generated subsets.

Result

Thus, the **subset generation problem was successfully implemented in Python**. For $E = [2, 3, 4, 5]$ and $x = 3$, the program generates all **8 subsets containing the element 3**.

Q17 – Universal Strings

Aim

To find all the strings in words1 that contain **every character, including its required frequency**, from every word in words2.

Algorithm

1. For every word in words2, count the frequency of each character.
2. Find the maximum required frequency for every character.
3. For each word in words1, count its character frequencies.
4. Check whether it contains every required character with at least the required frequency.
5. If yes, add the word to the result.
6. Return all universal strings.

Python Source Code

```
from collections import Counter

def word_subsets(words1, words2):
    required = Counter()

    for word in words2:
        count = Counter(word)
        for char in count:
            required[char] = max(required[char], count[char])

    result = []

    for word in words1:
        count = Counter(word)
        valid = True

        for char in required:
            if count[char] < required[char]:
                valid = False
                break

        if valid:
            result.append(word)

    return result

words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
words2 = ["e", "o"]

result = word_subsets(words1, words2)

print("Universal strings:", result)
```

```
===== RESTART: C:/Users/HP/Downloads/Documents/e
Universal strings: ['facebook', 'google', 'leetcode']
>|
```

Time Complexity

$$O(N \times L + M \times K)$$

where:

- N = number of words in words1
- L = maximum length of a word in words1
- M = number of words in words2
- K = maximum length of a word in words2

Space Complexity

$O(1)$ auxiliary space for the character-frequency tables, assuming a fixed alphabet.

Result

Thus, the **Universal Strings problem was successfully implemented using Python**. The program correctly identifies all strings in words1 that satisfy the character-frequency requirements of every word in words2.

Q18 – Maximum Probability Path

Aim

To find the path with the **maximum probability of success** between the given start and end nodes in an undirected weighted graph.

Algorithm

1. Create an adjacency list for the graph.
2. Store the success probability for each edge.
3. Set the probability of the start node to 1.0.
4. Use a priority queue to process the node having the highest probability.
5. For each neighboring node, calculate the new probability.
6. Update the neighbor if the new probability is greater than its current probability.
7. Continue until the end node is reached or all possible nodes are processed.
8. Return the maximum probability.
9. If the end node cannot be reached, return 0.

Python Source Code

File Edit Format Run Options Window Help

```
import heapq

def max_probability(n, edges, succProb, start, end):
    graph = [[] for _ in range(n)]

    for i in range(len(edges)):
        a, b = edges[i]
        p = succProb[i]
        graph[a].append((b, p))
        graph[b].append((a, p))

    probability = [0.0] * n
    probability[start] = 1.0

    pq = [(-1.0, start)]

    while pq:
        current_prob, node = heapq.heappop(pq)
        current_prob = -current_prob

        if node == end:
            return current_prob

        if current_prob < probability[node]:
            continue

        for neighbor, edge_prob in graph[node]:
            new_prob = current_prob * edge_prob

            if new_prob > probability[neighbor]:
                probability[neighbor] = new_prob
                heapq.heappush(pq, (-new_prob, neighbor))

    return 0.0

n = 3
edges = [[0, 1], [1, 2], [0, 2]]
succProb = [0.5, 0.5, 0.2]
start = 0
end = 2
```

```
===== RESTART: C:/User:
Maximum Probability: 0.25
> |
```

Time Complexity

$$O((V + E) \log V)$$

where V is the number of vertices and E is the number of edges.

Space Complexity

$$O(V + E)$$

Result

Thus, the **maximum probability path was successfully found using Python**. For the first example, the maximum success probability from node 0 to node 2 is **0.25**.

Q19 – Unique Paths

Aim

To find the number of possible unique paths for a robot to travel from the **top-left corner** to the **bottom-right corner** of an $m \times n$ grid, where the robot can move only **right or down**.

Algorithm

1. Create a DP table of size $m \times n$.
2. Initialize the first row and first column with 1.
3. For every remaining cell, calculate:
$$dp[i][j] = dp[i-1][j] + dp[i][j-1]$$
4. The value at the bottom-right cell is the total number of unique paths.
5. Return the result.

Python Source Code

```
def unique_paths(m, n):  
    dp = [[1] * n for _ in range(m)]  
  
    for i in range(1, m):  
        for j in range(1, n):  
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]  
  
    return dp[m - 1][n - 1]
```

```
m = 3  
n = 7  
  
result = unique_paths(m, n)  
  
print("Number of unique paths:", result)
```

```
===== RESTART: C:/Users/HI  
Number of unique paths: 28  
=
```

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

Result

Thus, the **Unique Paths** problem was successfully implemented using **Dynamic Programming in Python**. For $m = 3$ and $n = 7$, the total number of unique paths is **28**.

Q20 – Number of Good Pairs

Aim

To find the number of **good pairs** (i, j) in an integer array where:

- $\text{nums}[i] == \text{nums}[j]$
- $i < j$

Algorithm

1. Create an empty dictionary to store the frequency of each number.
2. Initialize pairs = 0.
3. Traverse the array from left to right.
4. For each number:
 - Add its current frequency to pairs.
 - Increase its frequency by 1.
5. Print the total number of good pairs.

Python Source Code

```
File Edit Format Run Options Window Help
def num_good_pairs(nums):
    frequency = {}
    pairs = 0

    for num in nums:
        if num in frequency:
            pairs += frequency[num]
            frequency[num] += 1
        else:
            frequency[num] = 1

    return pairs

nums = [1, 2, 3, 1, 1, 3]
result = num_good_pairs(nums)
print("Number of good pairs:", result)

===== RESTART: C:/Users/HF
Number of good pairs: 4
```

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Result

Thus, the **Good Pairs problem** was successfully implemented using **Python**. The program correctly counts all pairs having equal values with the first index smaller than the second.

Q21 – Find the City With the Smallest Number of Reachable Cities

Aim

To implement **Floyd-Warshall Algorithm** to find the shortest distance between every pair of cities and return the city having the **smallest number of reachable cities** within the given distance threshold. If there is a tie, return the city with the **greatest number**.

Algorithm

1. Create a distance matrix for all cities.
2. Set the distance from a city to itself as 0.
3. Store the given edge weights in the matrix.
4. Initialize unconnected cities with infinity.
5. Apply Floyd-Warshall Algorithm:
 - Consider each city as an intermediate city.
 - Update the shortest distance between every pair.
6. For each city, count the number of other cities whose shortest distance is less than or equal to distanceThreshold.
7. Select the city with the smallest count.
8. If two or more cities have the same count, select the city with the greater index.
9. Print the selected city.

Python Source Code

```
def find_city(n, edges, distanceThreshold):
    INF = float('inf')

    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    result = -1
    min_count = float('inf')

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= distanceThreshold:
                count += 1

        if count <= min_count:
            min_count = count
            result = i

    return result

n = 4
edges = [
    [0, 1, 3],
    [1, 2, 1],
    [1, 3, 4],
    [2, 3, 1]
]

distanceThreshold = 4

result = find_city(n, edges, distanceThreshold)

print("City with the smallest number of reachable cities:", result)
```

Time Complexity **$O(n^3)$** **Space Complexity** **$O(n^2)$** **Result**

Thus, the **Floyd-Warshall Algorithm** was successfully implemented in Python to determine the city with the minimum number of reachable cities within the given distance threshold. For the first test case, the answer is **City 3**.

Q22 – Network Delay Time**Aim**

To find the **minimum time required for all nodes to receive a signal** sent from a given starting node k using **Dijkstra's Algorithm**.

Algorithm

1. Create an adjacency list for the directed graph.
2. Store the travel time for each directed edge.
3. Initialize the distance of every node to infinity.
4. Set the distance of the starting node k to 0.
5. Use a priority queue to select the node with the smallest known distance.
6. Visit all neighboring nodes.
7. Update the distance if a shorter path is found.
8. Continue until all possible nodes are processed.
9. Find the maximum shortest distance.
10. If any node is unreachable, return -1; otherwise return the maximum distance.

Python Source Code

```
import heapq

def network_delay_time(times, n, k):
    graph = [[] for _ in range(n + 1)]

    for u, v, w in times:
        graph[u].append((v, w))

    distance = [float('inf')] * (n + 1)
    distance[k] = 0

    pq = [(0, k)]

    while pq:
        current_distance, node = heapq.heappop(pq)

        if current_distance > distance[node]:
            continue

        for neighbor, weight in graph[node]:
            new_distance = current_distance + weight

            if new_distance < distance[neighbor]:
                distance[neighbor] = new_distance
                heapq.heappush(pq, (new_distance, neighbor))

    maximum = max(distance[1:])

    if maximum == float('inf'):
        return -1

    return maximum
```

```
times = [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]
```

```
===== RESTART: C:/U
Network Delay Time: 2
```


Time Complexity

$O((V + E) \log V)$

Space Complexity

$O(V + E)$

Result

Thus, the **Network Delay Time problem was successfully implemented using Dijkstra's Algorithm in Python**. The program finds the minimum time required for all reachable nodes to receive the signal and returns -1 when some node cannot be reached.